KRISHISYNC AI | VIBE CODING SERIES | 2 OF 5

System Architecture

Four layers, data model, API surface and the prompts to scaffold them

Market-Aware Agri-Coordination & Smart Energy Optimization for Indian Smallholders

Challenge 1: Sustainable Agriculture & Resource Optimization

System Architecture

KrishiSync is a four-layer open-source stack. Each layer is small enough for one teammate to own and for an AI assistant to scaffold in a single session.

Layer	Technology	Responsibility
01 Frontend	React (Vite), react-i18next, Leaflet, Recharts, PWA	Localised farmer UI, glut heat-map, calendar, energy panel
02 Backend	Python FastAPI, Pydantic, SQLAlchemy, APScheduler	Heavy decision algorithms, REST API, background jobs
03 Data	PostgreSQL + PostGIS, Redis	Regional coordinates and radius queries, caching, job queue
04 Hardware	ESP32 or Arduino, capacitive soil probes	Soil moisture and pump-state telemetry

Data flow

```
ESP32 probe --HTTPS JSON--> FastAPI /sensors/readings --> PostgreSQL (sensor_readings)
                                        |
Weather API --(cron, cached)--> Redis +--> Irrigation Scheduler --> pump_schedules
Mandi prices --(cron)--> market_prices -+
Farm registrations --> PostGIS radius query --> Anti-Glut Engine --> alerts
                                        |
React PWA <------ REST (JSON, i18n keys) ------+
```

Suggested repository layout

```
krishisync/
  frontend/ React app (src/pages, src/components, src/i18n)
  backend/
    app/main.py FastAPI entry
    app/api/ routes only, no business logic
    app/engines/ antiglut.py, irrigation.py, advisory.py, energy.py (pure functions)
    app/models/ SQLAlchemy + PostGIS models
    app/jobs/ scheduled fetchers (weather, prices)
    app/config.py all thresholds and weights
    tests/ pytest, one file per engine
  firmware/ esp32_probe.ino
  data/seed/ demo farms, crops, prices (CSV)
  docker-compose.yml postgres+postgis, redis, backend, frontend
```

Core database tables (proposed)

Table	Key columns	Notes
farmers	id, name, phone, language	Phone-first identity; no email required
farms	id, farmer_id, location (geography Point), area_ha, terrain, soil_type, slope_pct, pump_type	GiST index on location
crops	id, name, season, water_need_mm, root_depth_mm, yield_t_per_ha, days_to_harvest	Seeded reference data

Table	Key columns	Notes
crop_plans	id, farm_id, crop_id, sowing_date, expected_harvest_date, area_ha	What each farm intends to plant
market_prices	id, crop_id, mandi, date, modal_price, arrivals_t	Seeded or fetched
regional_capacity	id, region, crop_id, window_start, window_end, absorb_t	Demand a region can absorb
sensor_readings	id, farm_id, ts, moisture_pct, n, p, k, pump_on, watts	Time-series, index on (farm_id, ts)
pump_schedules	id, farm_id, start_ts, duration_min, reason, status	Written by scheduler
tariff_slots	id, region, start_hour, end_hour, kind (peak/off-peak/solar)	Config-driven
alerts	id, farm_id, kind, severity, message_key, payload, created_at	Message keys, not raw text, for i18n

API surface (proposed)

Method and path	Purpose
POST /auth/otp/request, /auth/otp/verify	Phone OTP login
POST /farms	Register land location and terrain profile
GET /farms/{id}/glut-map?radius_km=20	Saturation per crop with colour band
GET /farms/{id}/recommendations	Ranked crop alternatives with reasons
GET /farms/{id}/calendar	Seeding, watering and nutrient timeline
POST /sensors/readings	ESP32 telemetry (device API key)
GET /farms/{id}/energy	Pump schedule, off-peak share, idle-load flags
POST /farms/{id}/pump/recompute	Force scheduler re-run
GET /alerts?farm_id=	Glut, rain-pause and idle-load alerts

Sensor payload

```
POST /sensors/readings header: X-Device-Key: <key>
{ "farm_id": 12, "ts": "2026-10-14T06:30:00Z",
  "moisture_pct": 21.4, "n": 38, "p": 14, "k": 120,
  "pump_on": false, "watts": 7.2 }
```

Data-source caveat. Weather (for example Open-Meteo) is freely available. Live mandi prices and real neighbour crop registrations may not be accessible during a hackathon, so build behind an interface and ship seeded CSV data first. Verify any public API's terms and availability before relying on it.

Architecture Vibe Prompts

Paste Master Prompt (PDF 1) first, then these in order. Run and test after each one.

VIBE PROMPT 1 | Scaffold the monorepo

```
Create the repo layout from the Architecture PDF. Add docker-compose.yml with
postgis/postgis, redis, backend (FastAPI + uvicorn --reload) and frontend (Vite).
Add a /health endpoint that checks Postgres and Redis. Add .env.example and a README
with the exact commands to run everything. Do not build features yet.
```

VIBE PROMPT 2 | Database schema and seed data

```
Using SQLAlchemy + GeoAlchemy2, create models and an Alembic migration for: farmers,
farms (geography Point, GiST index), crops, crop_plans, market_prices,
regional_capacity, sensor_readings, pump_schedules, tariff_slots, alerts.
Write scripts/seed.py that inserts ~12 crops, 60 demo farms clustered around 3 villages
(random points within a few km), crop plans with deliberate over-concentration in one
crop, 30 days of mock prices, and a peak/off-peak tariff. Print a summary of rows
inserted. Show me the schema before writing migration code.
```

VIBE PROMPT 3 | Farm registration and radius query

```
Implement POST /farms and a repository function farms_within_radius(lat, lon, radius_m)
using ST_DWithin on geography. Add pytest tests with the seeded data: a farm inside 20 km
is counted, one outside is not. Return only aggregated counts to clients, never other
farmers' identities.
```

VIBE PROMPT 4 | Sensor ingestion

```
Implement POST /sensors/readings with Pydantic validation, X-Device-Key auth, and
rejection of out-of-range values (moisture 0-100). Store to sensor_readings. Add
GET /farms/{id}/readings?hours=24. Write a Python script that simulates one ESP32
posting a reading every 10 seconds with a slow drying trend, so I can demo without hardware.
```

VIBE PROMPT 5 | Background jobs and caching

```
Add APScheduler jobs: fetch weather forecast every 3 hours into Redis (TTL 3h) behind a
WeatherProvider interface with an Open-Meteo implementation and a MockProvider; refresh
market_prices from a PriceProvider interface (seed CSV implementation first). Cache glut
results per (region, crop) in Redis for 1 hour. Every external call needs a timeout and
a graceful fallback to the last cached value.
```

VIBE PROMPT 6 | ESP32 firmware

```
Write an Arduino-framework sketch for ESP32: read a capacitive soil moisture sensor on an
analog pin, map raw ADC to 0-100% using two calibration constants, connect to WiFi, and POST
the JSON payload from the Architecture PDF every 15 minutes with retry and deep sleep between
readings. Put WiFi creds and the device key in a separate secrets.h. Explain the wiring and how
to calibrate dry and wet readings.
```